

Security Audit Report: VM Host Critical Vulnerabilities

Report ID: MX-VM-V1.4-AUDIT-001

Project: mx-chain-vm-v1_4-go

1. Executive Summary

This report documents critical security vulnerabilities identified in the WebAssembly (WASM) Virtual Machine host implementation (mx-chain-vm-v1_4-go). These vulnerabilities reside in the host-hook layer (EEI) and can be exploited to crash validator nodes or halt the network.

Four real vulnerabilities were identified. Two have been fully remediated. Two are pending remediation.

2. Vulnerability Details

2.1 [REAL-4] Unbounded BigInt Shift Left (Host OOM)

Attribute	Details
Severity	CRITICAL
Location	vmhost/vmhooks/bigIntOps.go:v1_4_bigIntShl
Impact	Host Process Termination (OOM)
Remediation	RESOLVED

Technical Analysis

The v1_4_bigIntShl hook exports the BigInt shift-left operation to the WASM guest. It accepts an int32 value for the number of bits to shift.

```
// Location: vmhost/vmhooks/bigIntOps.go
func v1_4_bigIntShl(context unsafe.Pointer, destinationHandle, opHandle int32, bits int32) {
    // ... gas charged for base op only ...
    dest.Lsh(a, uint(bits)) // VULNERABILITY: Unbounded allocation pre-patch
    managedType.ConsumeGasForBigIntCopy(dest)
}
```

If a contract provides a bits value of 2,147,483,647 (MaxInt32), the math/big library attempts to allocate a buffer large enough to hold 2 billion bits (~256 MB), crashing the node with OOM before gas is deducted.

Recommended Remediation

Add a hard ceiling on the bits parameter and charge gas pre-emptively based on the prospective result byte size before calling dest.Lsh:

```
if bits > 512000 {
    _ = vmhost.WithFault(vmhost.ErrNotEnoughGas, context, runtime.BigIntAPIErrorShouldFailExecution())
    return
}
prospectiveByteLen := (uint64(a.BitLen()) + uint64(bits)) / 8
managedType.ConsumeGasForThisIntNumberOfBytes(int(prospectiveByteLen))
dest.Lsh(a, uint(bits))
```

Remediation Applied

A hard ceiling of 512,000 bits was added. Gas is now charged pre-emptively based on the prospective result size before `dest.Lsh` executes.

2.2 [REAL-6] Unmetered Large Memory Allocation (OOM/DoS)

Attribute	Details
Severity	HIGH
Location	vmhost/vmhooks/baseOps.go:getArgumentsFromMemory
Impact	Host Memory Exhaustion / CPU Spike
Remediation	RESOLVED

Technical Analysis

The `getArgumentsFromMemory` helper loads complex argument arrays from WASM linear memory into the host before gas is deducted.

```
// Location: vmhost/vmhooks/baseOps.go
func getArgumentsFromMemory(host vmhost.VMHost, numArguments int32, ...) {
    argumentsLengthData, _ := runtime.MemLoad(argumentsLengthOffset, numArguments*4) // Unmetered
    data, _ := runtime.MemLoadMultiple(dataOffset, argumentLengths) // Unmetered
    return data, totalArgumentBytes, nil
    // Gas deducted by caller AFTER return -- too late
}
```

An attacker can specify huge argument arrays to exhaust host memory before any gas is charged.

Recommended Remediation

Implement gas-first loading: compute prospective allocation sizes and deduct gas **before** any `MemLoad`, failing closed if gas is insufficient:

```
// Charge for metadata array before loading
metaGas := math.MulUint64(uint64(numArguments)*4, metering.GasSchedule().BaseOperationCost.DataCopyPerByte)
metering.UseGas(metaGas)
if metering.GasLeft() == 0 {
```

```
    return nil, 0, vmhost.ErrNotEnoughGas
}
// Then load metadata, compute total data size, charge for data payload before loading
```

Remediation Applied

Gas-first loading: gas is now pre-emptively checked and deducted for both the metadata slice and the argument data slice before any memory allocation occurs.

2.3 [REAL-7] ESDT Nil Pointer Dereference (Consensus-Halting DoS)

Attribute	Details
Severity	CRITICAL
Location	vmhost/vmhooks/baseOps.go:v1_4_getESDTBalance, v1_4_getESDTTokenData
Impact	Deterministic nil pointer panic — crashes all validator nodes validating the block
Remediation	PENDING

Technical Analysis

Both v1_4_getESDTBalance and v1_4_getESDTTokenData call getESDTDataFromBlockchainHook, which delegates to blockchain.GetESDTToken. When queried for a non-existent token or an account with no ESDT balance, the hook returns (nil, nil) — no error, but a nil pointer.

```
// Location: vmhost/vmhooks/baseOps.go
esdtData, err := getESDTDataFromBlockchainHook(...)
if vmhost.WithFault(err, ...) { return -1 } // err is nil, so this passes

// CRASH VECTOR: esdtData is nil here
err = runtime.MemStore(resultOffset, esdtData.Value.Bytes()) // nil dereference PANIC
```

And in v1_4_getESDTTokenData:

```
value := managedType.GetBigIntOrCreate(valueHandle)
value.Set(esdtData.Value) // nil dereference PANIC
```

Note: Sibling hooks such as v1_4_getESDTNFTNameLength already correctly guard against this condition.

Exploit Scenario

- Attacker deploys a contract that calls getESDTBalance with a non-existent token ID.
- GetESDTToken returns (nil, nil) — no error.
- Host dereferences esdtData.Value — runtime panic.
- Every validator processing this block crashes on the same transaction.
- Result: **Network halt.**

Recommended Remediation

```
if esdtData == nil || esdtData.Value == nil {
    vmhost.WithFaultIfFailAlwaysActive(vmhost.ErrNilESDTData, context,
        runtime.BaseOpsErrorShouldFailExecution())
    return 0
}
```

2.4 [REAL-8] ESDT Roles Parser Index Out of Bounds (State-Triggered Host Panic)

Attribute	Details
Severity	HIGH
Location	vmhost/vmhooks/eei_helpers.go:getESDTRoles
Impact	Unhandled index out of range panic — crashes validator nodes
Remediation	PENDING

Technical Analysis

The `getESDTRoles` function parses a custom binary format read directly from blockchain storage. It makes no bounds checks on the byte array, trusting storage to always be well-formed.

```
// Location: vmhost/vmhooks/eei_helpers.go
func getESDTRoles(dataBuffer []byte) int64 {
    for currentIndex < valueLen {
        currentIndex += 1                // skip \n

        roleLen := int(dataBuffer[currentIndex]) // CRASH: out of bounds if buffer ends
here
        currentIndex += 1

        endIndex := currentIndex + roleLen
        roleName := dataBuffer[currentIndex:endIndex] // CRASH: slice out of bounds
        currentIndex = endIndex
    }
}
```

If storage returns a truncated, empty, or malformed byte buffer — due to state corruption, incomplete writes, or protocol migration — this parser triggers a runtime error: index out of range panic.

Recommended Remediation

```
for currentIndex < valueLen {
    if currentIndex+1 > valueLen { break }
    currentIndex += 1

    if currentIndex+1 > valueLen { break }
    roleLen := int(dataBuffer[currentIndex])
    currentIndex += 1

    endIndex := currentIndex + roleLen
```

```
    if endIndex > valueLen { break }
    roleName := dataBuffer[currentIndex:endIndex]
    currentIndex = endIndex
    result |= roleFromByteArray(roleName)
}
```

2.5 [REAL-9] Managed ESDT API Nil Pointer Dereference (Consensus-Halting DoS)

Attribute	Details
Severity	CRITICAL
Location	vmhost/vmhooks/managedei.go:v1_4_managedGetESDTBalance, v1_4_managedGetESDTTokenData
Impact	Deterministic nil pointer panic — crashes all validator nodes validating the block
Remediation	PENDING

Technical Analysis

The managed API variants of the ESDT balance and token data hooks suffer the **same nil dereference vulnerability** as REAL-7, but via the modern managed API path used by all contracts compiled with the latest MultiversX SDK.

```
// Location: vmhost/vmhooks/managedei.go
esdtToken, err := blockchain.GetESDTToken(address, tokenID, uint64(nonce))
if err != nil {
    _ = vmhost.WithFault(...)
    return
}
// No nil check – GetESDTToken can return (nil, nil)
value.Set(esdtToken.Value) // PANIC: nil pointer dereference
```

And in v1_4_managedGetESDTTokenData:

```
value.Set(esdtToken.Value) // PANIC if esdtToken == nil
managedType.SetBytes(propertiesHandle, esdtToken.Properties) // PANIC if esdtToken == nil
```

Unlike REAL-7 (legacy flat API), modern contracts compiled with the current MultiversX Rust SDK call the **managed API** exclusively, making this variant **more frequently reachable** in a production network.

Recommended Remediation

```
if esdtToken == nil || esdtToken.Value == nil {
    vmhost.WithFaultIfFailAlwaysActive(vmhost.ErrNilESDTData, context,
        runtime.BaseOpsErrorShouldFailExecution())
    return
}
```

2.6 [REAL-10] Partial Nil Guard in BigInt ESDT External Balance Hook

Attribute	Details
Severity	HIGH
Location	vmhost/vmhooks/bigIntOps.go:v1_4_bigIntGetESDTExternalBalance
Impact	Nil pointer panic on esdtData.Value — crashes validator nodes
Remediation	PENDING

Technical Analysis

v1_4_bigIntGetESDTExternalBalance has an incomplete nil guard. It correctly checks if the esdtData struct pointer is nil, but does not check if the value field within the struct is nil before dereferencing it.

```
// Location: vmhost/vmhooks/bigIntOps.go
if esdtData == nil { // ✅ struct nil check present
    return
}

value := managedType.GetBigIntOrCreate(resultHandle)
value.Set(esdtData.Value) // ❌ PANIC: esdtData.Value can still be nil
```

The esdt.ESDDigitalToken struct may have a non-nil pointer with a nil value field when an account exists but has a zero or uninitialised token balance. This passes the struct nil guard but panics on field access.

Recommended Remediation

```
if esdtData == nil || esdtData.Value == nil {
    return
}
```

3. Remediation Summary

ID	Title	Severity	Status
REAL-4	Unbounded BigInt Shift Left (OOM)	CRITICAL	RESOLVED
REAL-6	Unmetered Memory Allocation (OOM/DoS)	HIGH	RESOLVED
REAL-7	ESDT Nil Pointer Dereference — Legacy API (Consensus Halt)	CRITICAL	PENDING
REAL-8	ESDT Roles Parser Out-of-Bounds (Node Panic)	HIGH	PENDING
REAL-9	ESDT Nil Pointer Dereference — Managed API (Consensus Halt)	CRITICAL	PENDING
REAL-10	Partial Nil Guard in BigInt ESDT Balance Hook	HIGH	PENDING

